北京師範大學
BEIJING NORMAL UNIVERSITY

计算机图形学 实验报告

姓名 杨博文　　学号 202311081015
院系 人工智能学院　　专业 计算机科学与技术

2025 年 5 月 9 日

摘　要

计算机图形学的第六次实验，主要内容包括为环境加载与几何图元。

目录

1 运行环境及运行方式

硬件为 Macbook Air，M2 芯片，操作系统为 MacOS Sequoia 15.0。

IDE 为 Xcode Version16.0。

使用 GLFW+GLAD，程序在 Xcode 内编译运行。

2 任务 1: 天空与地板

2.1 任务要求

基于实验五的任务 3，结合“depth_testint.cpp”与”cubemaps_skybox.cpp”，为 backpack 模型增加简单的天空盒背景与带纹理的地板；

2.2 关键代码

```
//首先，着色器:
    Shader floorshader("/Users/yangbowen/Desktop/本科/大二/大二下课程
    /计算机图形学/LearnOpenGL-master/src/4.advanced_opengl/1.1.depth_testing
    /1.1.depth_testing.vs", "/Users/yangbowen/Desktop/本科/大二
    /大二下课程/计算机图形学/LearnOpenGL-master/src/4.advanced_opengl/1.1.depth_testi
    /1.1.depth_testing.fs");
    Shader skyboxShader("/Users/yangbowen/Desktop/本科/大二/大二下课程
    /计算机图形学/LearnOpenGL-master/src/4.advanced_opengl/6.1.cubemaps_skybox
    /6.1.skybox.vs", "/Users/yangbowen/Desktop/本科/大二/大二下课程
    /计算机图形学/LearnOpenGL-master/src/4.advanced_opengl/
    6.1.cubemaps_skybox/6.1.skybox.fs");
//然后，点定义:
    float planeVertices[] = {
        // positions          // texture Coords (note we set
        these higher than 1 (together with GL_REPEAT as texture
        wrapping mode). this will cause the floor texture to repeat)
         5.0f, -0.5f,  5.0f,  2.0f, 0.0f,
        -5.0f, -0.5f,  5.0f,  0.0f, 0.0f,
        -5.0f, -0.5f, -5.0f,  0.0f, 2.0f,

         5.0f, -0.5f,  5.0f,  2.0f, 0.0f,
        -5.0f, -0.5f, -5.0f,  0.0f, 2.0f,
         5.0f, -0.5f, -5.0f,  2.0f, 2.0f
```

```
};
float skyboxVertices[] = {
    // positions
    -1.0f,  1.0f, -1.0f,
    -1.0f, -1.0f, -1.0f,
     1.0f, -1.0f, -1.0f,
     1.0f, -1.0f, -1.0f,
     1.0f,  1.0f, -1.0f,
    -1.0f,  1.0f, -1.0f,

    -1.0f, -1.0f,  1.0f,
    -1.0f, -1.0f, -1.0f,
    -1.0f,  1.0f, -1.0f,
    -1.0f,  1.0f, -1.0f,
    -1.0f,  1.0f,  1.0f,
    -1.0f, -1.0f,  1.0f,

     1.0f, -1.0f, -1.0f,
     1.0f, -1.0f,  1.0f,
     1.0f,  1.0f,  1.0f,
     1.0f,  1.0f,  1.0f,
     1.0f,  1.0f, -1.0f,
     1.0f, -1.0f, -1.0f,

    -1.0f, -1.0f,  1.0f,
    -1.0f,  1.0f,  1.0f,
     1.0f,  1.0f,  1.0f,
     1.0f,  1.0f,  1.0f,
     1.0f, -1.0f,  1.0f,
    -1.0f, -1.0f,  1.0f,

    -1.0f,  1.0f, -1.0f,
     1.0f,  1.0f, -1.0f,
     1.0f,  1.0f,  1.0f,
     1.0f,  1.0f,  1.0f,
    -1.0f,  1.0f,  1.0f,
    -1.0f,  1.0f, -1.0f,
```

```
        -1.0f, -1.0f, -1.0f,
        -1.0f, -1.0f,  1.0f,
         1.0f, -1.0f, -1.0f,
         1.0f, -1.0f, -1.0f,
        -1.0f, -1.0f,  1.0f,
         1.0f, -1.0f,  1.0f
    };
//第三, VAO和VBO:

    // plane VAO
    unsigned int planeVAO, planeVBO;
    glGenVertexArrays(1, &planeVAO);
    glGenBuffers(1, &planeVBO);
    glBindVertexArray(planeVAO);
    glBindBuffer(GL_ARRAY_BUFFER, planeVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(planeVertices)
    , &planeVertices, GL_STATIC_DRAW);
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    5 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(1);
    glVertexAttribPointer(1, 2, GL_FLOAT, GL_FALSE,
    5 * sizeof(float), (void*)(3 * sizeof(float)));
    glBindVertexArray(0);

    // skybox VAO
    unsigned int skyboxVAO, skyboxVBO;
    glGenVertexArrays(1, &skyboxVAO);
    glGenBuffers(1, &skyboxVBO);
    glBindVertexArray(skyboxVAO);
    glBindBuffer(GL_ARRAY_BUFFER, skyboxVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(skyboxVertices),
    &skyboxVertices, GL_STATIC_DRAW);
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
    3 * sizeof(float), (void*)0);
```

```
//第四, load Texture:
    // load textures
    // -------------
    unsigned int floorTexture = loadTexture("/Users
    /yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
    /LearnOpenGL-master/resources/textures/metal.png");

    vector<std::string> faces
    {
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/right.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/left.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/top.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/bottom.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/front.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/back.jpg"
    };
    unsigned int cubemapTexture = loadCubemap(faces);

    floorshader.use();
    floorshader.setInt("texture1", 0);

    skyboxShader.use();
    skyboxShader.setInt("skybox", 0);
//第五, 在循环中加入着色器和VAO的使用:
        // floor
        floorshader.use();
        floorshader.setMat4("view", view);
        floorshader.setMat4("projection", projection);
        glBindVertexArray(planeVAO);
        glBindTexture(GL_TEXTURE_2D, floorTexture);
        floorshader.setMat4("model", glm::mat4(1.0f));
```

```
        glDrawArrays(GL_TRIANGLES, 0, 6);
        glBindVertexArray(0);

        //skybox cube
        glDepthFunc(GL_LEQUAL);  // change depth function so depth
        test passes when values are equal to depth buffer's content
        view = glm::mat4(glm::mat3(camera.GetViewMatrix()));
        // remove translation from the view matrix
        skyboxShader.use();
        skyboxShader.setMat4("view", view);
        skyboxShader.setMat4("projection", projection);
        glBindVertexArray(skyboxVAO);
        glActiveTexture(GL_TEXTURE0);
        glBindTexture(GL_TEXTURE_CUBE_MAP, cubemapTexture);
        glDrawArrays(GL_TRIANGLES, 0, 36);
        glBindVertexArray(0);
        glDepthFunc(GL_LESS); // set depth function back to default
//最后，补充函数定义：

// utility function for loading a 2D texture from file
// ---------------------------------------------------
unsigned int loadTexture(char const *path)
{
    unsigned int textureID;
    glGenTextures(1, &textureID);

    int width, height, nrComponents;
    unsigned char *data = stbi_load(path, &width, &height,
    &nrComponents, 0);
    if (data)
    {
        GLenum format;
        if (nrComponents == 1)
            format = GL_RED;
        else if (nrComponents == 3)
            format = GL_RGB;
        else if (nrComponents == 4)
```

```
            format = GL_RGBA;

        glBindTexture(GL_TEXTURE_2D, textureID);
        glTexImage2D(GL_TEXTURE_2D, 0, format, width,
        height, 0, format, GL_UNSIGNED_BYTE, data);
        glGenerateMipmap(GL_TEXTURE_2D);

        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_S, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_WRAP_T, GL_REPEAT);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MIN_FILTER,
        GL_LINEAR_MIPMAP_LINEAR);
        glTexParameteri(GL_TEXTURE_2D, GL_TEXTURE_MAG_FILTER, GL_LINEAR);

        stbi_image_free(data);
    }
    else
    {
        std::cout << "Texture failed to load at path: " << path << std::endl;
        stbi_image_free(data);
    }

    return textureID;
}
// loads a cubemap texture from 6 individual texture faces
// order:
// +X (right)
// -X (left)
// +Y (top)
// -Y (bottom)
// +Z (front)
// -Z (back)
// -------------------------------------------------------
unsigned int loadCubemap(vector<std::string> faces)
{
    unsigned int textureID;
    glGenTextures(1, &textureID);
    glBindTexture(GL_TEXTURE_CUBE_MAP, textureID);
```

```
    int width, height, nrChannels;
    for (unsigned int i = 0; i < faces.size(); i++)
    {
        unsigned char *data = stbi_load(faces[i].c_str(), &width,
        &height, &nrChannels, 0);
        if (data)
        {
            glTexImage2D(GL_TEXTURE_CUBE_MAP_POSITIVE_X + i, 0,
            GL_RGB, width, height, 0, GL_RGB, GL_UNSIGNED_BYTE, data);
            stbi_image_free(data);
        }
        else
        {
            std::cout << "Cubemap texture failed to load at
            path: " << faces[i] << std::endl;
            stbi_image_free(data);
        }
    }
    glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MIN_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_MAG_FILTER, GL_LINEAR);
    glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_S, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_T, GL_CLAMP_TO_EDGE);
    glTexParameteri(GL_TEXTURE_CUBE_MAP, GL_TEXTURE_WRAP_R, GL_CLAMP_TO_EDGE);

    return textureID;
}
```

主要分为六个部分，分别是着色器、点、VAO&VBO 定义、加载纹理、在循环中使用着色器和 VAO、补充实现 load 函数。代码清晰，不再解释。

2.3 结果图片

以上操作都做了之后，让我们看一看最终的结果。

图 1: 实验 1 结果

3 任务 2: 更合理的天空与地板

3.1 任务要求

基于任务 1，调整地面或模型的位置与大小，防止穿模并与场景适配，展示完整的正面静态场景；

3.2 关键代码

```
//第一，在加载部分改：
    stbi_set_flip_vertically_on_load(false);
    vector<std::string> faces
    {
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/right.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/left.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/top.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/bottom.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/front.jpg",
        "/Users/yangbowen/Desktop/本科/大二/大二下课程/计算机图形学
        /LearnOpenGL-master/resources/textures/skybox/back.jpg"
    };
```

```
    unsigned int cubemapTexture = loadCubemap(faces);
    stbi_set_flip_vertically_on_load(true);
//第二，对floor在循环内的部分进行改：
        // floor
        floorshader.use();
        floorshader.setMat4("view", view);
        floorshader.setMat4("projection", projection);
        glm::mat4 floorModel=glm::mat4(1.0f);
        floorModel = glm::translate(floorModel,
        glm::vec3(0.0f, -1.5f, 0.0f));
        float distance = glm::length(camera.Position);
        float scaleFactor = glm::clamp(distance / 10.0f, 1.0f, 5.0f);
        floorModel = glm::scale(floorModel,
        glm::vec3(scaleFactor, 1.0f, scaleFactor));

        floorshader.setMat4("model", floorModel);
        glBindVertexArray(planeVAO);
        glBindTexture(GL_TEXTURE_2D, floorTexture);
        glDrawArrays(GL_TRIANGLES, 0, 6);
        glBindVertexArray(0);
```

我们要改两部分，在加载天空盒前后分别修改设置为 false 和 true 来避免天空盒被颠倒，在循环中修改 floor 部分代码使得地板根据视角变大而相应放大。

3.3 结果图片

进行上述两个修改后，得到结果如下图。

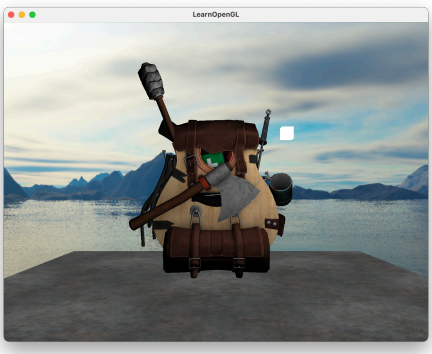

图 2: 实验 2 结果

4 任务 3: 增加小草

4.1 任务要求

基于任务 2，结合” geometry_shader_houses.cpp”，添加平面小草作为地面装饰，并展示完整的多视角静态场景（至少 3 个不同视角）。

4.2 关键代码

main.cpp:

```
\\首先，增加着色器:
    Shader grassShader("/Users/yangbowen/Desktop/本科/大二
    /大二下课程/计算机图形学/2025.5.8计算机图形学/Week10：小草小草
    /Week10：小草小草/9.1.geometry_shader.vs", "/Users
    /yangbowen/Desktop/本科/大二/大二下课程/计算机图形学/2025.5.8计算机图形学
    /Week10：小草小草/Week10：小草小草/9.1.geometry_shader.fs", "/Users/yangbowen/Des
    /2025.5.8计算机图形学/Week10：小草小草/Week10：小草小草
    /9.1.geometry_shader.gs");
\\第二，补充点定义:
    float grasspoints[] = {
        //     x      y       z       r     g     b
        -4.0f, -1.6f, -4.0f,  0.2f, 0.8f, 0.3f,
         0.0f, -1.6f, -2.0f,  0.2f, 0.8f, 0.3f,
         2.0f, -1.6f, -1.0f,  0.2f, 0.8f, 0.3f,
         4.0f, -1.6f,  0.0f,  0.2f, 0.8f, 0.3f,
        -3.5f, -1.6f,  1.0f,  0.2f, 0.8f, 0.3f,
         1.5f, -1.6f,  2.0f,  0.2f, 0.8f, 0.3f
    };
\\第三，VAO和VBO:
    //grass VAO
    unsigned int grassVBO, grassVAO;
    glGenVertexArrays(1, &grassVAO);
    glGenBuffers(1, &grassVBO);
    glBindVertexArray(grassVAO);
    glBindBuffer(GL_ARRAY_BUFFER, grassVBO);
    glBufferData(GL_ARRAY_BUFFER, sizeof(grasspoints),
    grasspoints, GL_STATIC_DRAW);
    glVertexAttribPointer(0, 3, GL_FLOAT, GL_FALSE,
```

```
    6 * sizeof(float), (void*)0);
    glEnableVertexAttribArray(0);
    glVertexAttribPointer(1, 3, GL_FLOAT, GL_FALSE,
    6 * sizeof(float), (void*)(3 * sizeof(float)));
    glEnableVertexAttribArray(1);
    glBindVertexArray(0);
\\第四，在主循环中增加grass的相关内容：
        // grass
        grassShader.use();
        glm::mat4 grassmodel = glm::mat4(1.0f);
        grassShader.setMat4("model", grassmodel);
        grassShader.setMat4("view", view);
        grassShader.setMat4("projection", projection);
        glBindVertexArray(grassVAO);  // 使用绑定的小草图元 VAO
        glDrawArrays(GL_POINTS, 0, 6);
        glBindVertexArray(0);

9.1.geometry_shader.gs：
\\第五，改几何着色器，绘制小草
#version 330 core
layout (points) in;
layout (triangle_strip, max_vertices = 6) out;
\\这里代表着最大点的个数，需要改成6。

in VS_OUT {
    vec3 color;
} gs_in[];

out vec3 fColor;

void build_grass(vec4 position)
{
    fColor = gs_in[0].color;
    // 第一片叶子
    gl_Position = position + vec4(-0.15, -0.1, 0.0, 0.0);
    EmitVertex();
    gl_Position = position + vec4( 0.15, -0.1, 0.0, 0.0);
```

```
    EmitVertex();
    gl_Position = position + vec4( 0.1, 0.2, 0.0, 0.0);
    fColor = vec3(1.0, 1.0, 1.0);
    EmitVertex();
    EndPrimitive();

    fColor = gs_in[0].color;//恢复颜色
    // 第二片叶子
    gl_Position = position + vec4(-0.15, -0.1, 0.0, 0.0);
    EmitVertex();
    gl_Position = position + vec4(0.15, -0.1, 0.0, 0.0);
    EmitVertex();
    gl_Position = position + vec4(-0.1, 0.3, 0.0, 0.0);
    fColor = vec3(1.0, 1.0, 1.0);
    EmitVertex();
    EndPrimitive();
}

void main() {
    build_grass(gl_in[0].gl_Position);
}
```

9.1.geometry_shader.vs:

```
\\最后，我们修改vs，让aPos接收三维
#version 330 core
layout (location = 0) in vec3 aPos;
layout (location = 1) in vec3 aColor;

out VS_OUT {
    vec3 color;
} vs_out;

uniform mat4 model;
uniform mat4 view;
uniform mat4 projection;

void main()
```

```
{
    vs_out.color = aColor;
    gl_Position = projection * view * model * vec4(aPos, 1.0);
}
```

代码较长，主要内容包括：加载 shader，新建点，新建 vao、vbo，在主循环中使用 grass 的相关着色器和 vao、vbo，修改着色器 gs 和 vs。

4.3 结果图片

三张运行结果如下三图。

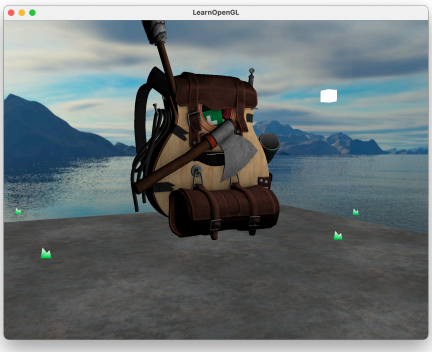

图 3: 实验 3 结果 1

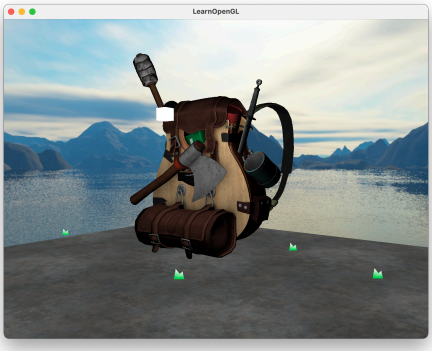

图 4: 实验 3 结果 2

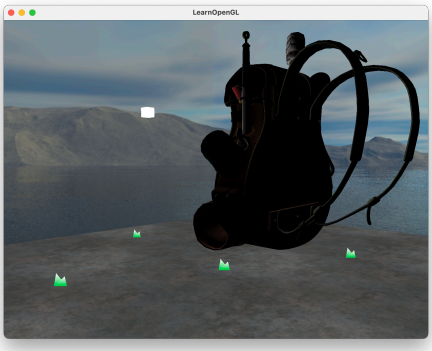

图 5: 实验 3 结果 3

5 题外话

有趣的实验。这次是真有趣。一步步看着东西从小小的看上去不太真实的变到加上了背景、灯光，吧啦吧啦。